



平顶山学院

PINGDINGSHAN UNIVERSITY

《面向对象程序设计实训》

题 目: SiteLens 网站安全透视系统
的设计与实现

院 (系): 信息工程学院

专业年级: 信息安全 2025 级

组 长: 程 一

成 员:

2026 年 09 月 06 日

摘要

随着互联网应用的快速迭代，现代网站普遍由框架、CDN、统计分析、支付接口等数十种第三方组件拼装而成，任何一个组件的历史漏洞都可能成为整个站点的攻击入口；与此同时，全球漏洞情报以每天数十条的速度增长，仅有效 CVE 编号已超过二十二万条，依靠人工翻查源码、记忆 CVE 的传统评估方式已难以为继。针对这一问题，本系统以信息安全专业的真实工作场景为依托，设计并实现了一个验证型 Web 漏洞扫描器——SiteLens 网站安全透视系统。

系统采用 Python 3.10 语言开发，基于 requests 与 BeautifulSoup 实现网站数据采集与解析，基于 Flask 与 PostgreSQL 实现扫描服务与数据持久化，运用封装、继承、多态、组合等面向对象技术构建了以 BaseDetector 检测器继承体系为核心的扫描引擎。系统实现了目标安全校验、网页证据采集、技术指纹识别（2850 余条指纹、78 个类别）、漏洞主动验证（2613 条社区模板与 42 条自研无害检测项）、漏洞情报三级判定（11024 条情报、OSV 版本区间比较、KEV 在野利用红标）、TLS 与 DNS 网络层检测、源码静态审计、登录爆破授权测试、批量扫描、历史对比与报告导出等功能，并通过零构建的 Web 工作台统一呈现。

系统对 SSRF 攻击、SQL 注入、路径穿越等风险做了针对性防护，全部数据库访问采用字面量 SQL 加参数绑定，扫描行为遵循全局限速、身份标识与 robots 协议，仅面向授权目标使用。经 49 项单元测试与 32 项接口冒烟测试验证，并通过对真实站点的多轮实测，各项功能正确可用，达到了预期的设计目标，具有较强的实用性与工程价值。

关键词：Python；面向对象；网络爬虫；漏洞扫描；信息安全

目 录

1 绪论	1
1.1 背景与意义	1
1.2 设计的主要思路及内容	2
1.3 项目分工	3
2 需求分析	3
2.1 功能需求分析	4
2.2 性能需求分析	6
2.3 运行环境与约束	7
3 总体设计	8
3.1 系统总体设计	8
3.2 数据库设计	11
3.3 界面设计原则	12
4 详细设计与实现	13
4.1 目标校验与扫描引擎主流程	14
4.2 数据采集与解析模块	15
4.3 技术指纹识别模块	16
4.4 验证型检测引擎	17
4.5 漏洞情报关联模块	20
4.6 网络层检测模块	22
4.7 登录爆破与源码审计模块	24
4.8 Web 工作台与数据持久化模块	25
4.9 系统部署与使用说明	28
5 系统测试	29
5.1 测试目的	29
5.2 测试用例	30
5.3 测试结论	33
6 总结	33

1 绪论

1.1 背景与意义

随着互联网应用规模的不断扩大，一个普通网站的背后往往运行着 Web 框架、CDN、统计分析、支付接口、字体库等数十种第三方组件。这些组件由不同社区以不同节奏维护，任何一个组件暴露出的历史漏洞，都可能成为攻击者进入整个站点的入口。以近年发生的安全事件为例，大量入侵的起点并不是目标业务代码本身的缺陷，而是站点引用的某个组件版本停留在已知漏洞的影响区间内。与此同时，全球漏洞情报以每天数十条的速度增长，仅有效 CVE 编号已超过二十二万条，CISAKEV 在野利用清单收录的漏洞也超过一千六百条。传统的网站安全评估依赖人工翻查网页源码、逐个比对响应头，再凭记忆联想相关漏洞，效率低、覆盖面窄、结论难复核，已经无法满足实际工作对时效性与准确性的要求。

本课题面向信息安全专业的真实工作场景，设计并实现了验证型 Web 漏洞扫描器 SiteLens 网站安全透视系统。系统的意义主要体现在三个方面：其一，把「打开开发者工具逐个猜」的手工分析过程，变为一次输入、全自动完成的标准化流程，显著提升了评估效率；其二，将识别结果与漏洞情报按版本区间自动比对，把「同名组件可能有漏洞」的粗放提示升级为「确认受影响 / 可能受影响 / 不受影响」的三级判定，有效抑制了误报；其三，全部扫描行为内置限速、身份标识与 SSRF 防护，仅面向授权目标，符合《中华人民共和国网络安全法》对授权测试的要求，可作为合法授权评估的教学与工程工具。

从行业现状看，商业扫描器功能全面但价格高昂、以云服务形态交付，扫描数据需要上传到第三方；单一功能的开源工具（如仅做端口扫描、仅做指纹识别）则要求使用者自行拼接流程。对于教学场景与中小团队的内部自查而言，一个「本地运行、数据不出机器、覆盖评估全流程」的工具是更契合的选择——这正是本系统在设计之初确立的产品位置。

对本实训而言，该项目覆盖了课程考核的全部核心知识点：扫描引擎以抽象基类与七个检测器子类体现了继承与多态，以实体类私有属性与只读访问器体现了封装，

以引擎对采集、解析、匹配等组件的持有关系体现了组合；数据采集环节完整运用了 requests 会话管理与 BeautifulSoup 结构化解析两项爬虫技术；数据库存储、异步任务、进度显示、扫描历史与报告导出等扩展能力则作为综合运用的加分项一并实现。

1.2 设计的主要思路及内容

系统总体思路是「先看清、再验证、后定论」。第一步看清目标：采集首页与同域子页面，从响应头、Cookie、meta 标签、DOM 结构、脚本地址等多类证据中识别网站技术栈；第二步主动验证：按识别出的技术路由社区验证模板与自研无害检测项，用真实请求证实漏洞是否存在；第三步给出结论：将验证结果与漏洞情报按版本区间比对，输出三级判定、安全响应头评分与中文修复建议，并将全部结果持久化以支持检索、对比与导出。三步之间以数据流水线衔接，每一步的产物都是下一步的输入，任一环节的结论都可以回溯到原始证据。

系统采用 B/S 架构与分层设计：展示层为零构建的手写 HTML/CSS/JS 页面；应用层为 Flask 实现的页面路由、REST API 与异步任务队列；核心层为 scanner 包，包含目标校验、采集解析、指纹检测、验证引擎、情报关联、网络层检测、源码审计等子模块；数据层为 PostgreSQL 18 数据库（sitelens 库），存储指纹知识库、漏洞情报与扫描结果。技术栈为 Python 3.10、Flask 3、psycpg2、requests、BeautifulSoup4、PyYAML 与 PostgreSQL 18。

开发过程按「核心引擎—验证能力—外围功能」三阶段推进：第一阶段实现目标校验、采集解析与指纹检测的主链路，打通「输入网址到输出技术清单」的最小闭环；第二阶段引入验证型检测引擎、Nuclei 社区模板转换器与情报三级判定，把系统从「看得清」升级为「验得准」；第三阶段完成 Web 工作台、批量扫描、扫描历史与 diff、情报库检索、报告导出与登录爆破等外围模块，并进行系统测试。每一阶段结束都以单元测试与真实站点实测做回归验证，保证增量开发不破坏已有能力。各阶段的目标、交付物与验收标准如表 1-1 所示。

表 1-1 开发阶段与里程碑

阶段	目标	主要交付物	验收标准
----	----	-------	------

阶段	目标	主要交付物	验收标准
第一阶段 核心引擎	打通扫描主链路	目标校验器、采集解析器、7 个检测器、指纹知识库入库管线	输入网址可输出技术清单，短词零误报
第二阶段 验证能力	从「看得清」到「验得准」	验证引擎、Nuclei 模板转换器（2613 条）、情报三级判定、KEV 红标	靶场样本全部命中，区间判定结论正确
第三阶段 外围功能	可用性与工程化	Web 工作台八个页面、批量/历史 diff/导出/API、登录爆破、网络层检测	全量测试通过，真实站点实测达标

工程实践上采用「小步提交 + 测试先行」的开发方式：核心逻辑先写单元测试再补实现，每完成一个模块即进行代码走查与静态安全扫描，发现问题当轮修复；所有对外行为（新增接口、变更导出格式）都必须同步更新文档与冒烟用例。这套节奏保证了三个阶段叠加开发后，系统仍然保持 49 项测试全绿的可交付状态。

1.3 项目分工

本项目由组长统一规划、分模块协作完成，分工如表 1-2 所示。

表 1-2 项目分工

成员	承担工作	占比
组长 程一	总体架构设计；扫描引擎与验证引擎核心编码；数据库模型与 REST API 设计；合规与安全防护方案	40%
成员（待填写）	数据采集与解析模块；指纹知识库整理与导入管线；Nuclei 模板转换器	20%
成员（待填写）	Web 前端工作台八个页面；情报库检索页；明暗双主题与交互细节	20%
成员（待填写）	测试用例设计与执行；真实站点实测；文档整理与答辩材料	20%

全部成员共同参与代码走查、合规审查与阶段评审。各模块职责边界清晰，接口由组长统一定义后并行开发，联调阶段每周合并一次并跑全量回归测试。

2 需求分析

系统在立项阶段对照课程考核目标与真实安全评估工作流，从功能与性能两个维度进行了需求分析，确保每一项功能都有明确的使用场景、输入输出与验收标准。需

求分析遵循两个原则：一是所有功能必须落在一个真实的安全评估动作上，不做无法落地的「演示功能」；二是所有主动能力必须可关闭、可审计，把合规当作需求而不是事后补丁。

2.1 功能需求分析

系统功能围绕「一次完整的安全评估」展开，共划分为八个功能模块，整体功能需求如表 2-1 所示。

表 2-1 系统功能需求一览

功能模块	需求描述	对应考核知识点
目标安全校验	对输入 URL 做协议白名单、主机黑名单与 DNS 解析逐 IP 校验，拒绝内网与保留地址	异常处理、字符串处理
数据采集与解析	requests 会话采集页面，BeautifulSoup 解析响应头、Cookie、meta、DOM、脚本等多类证据	requests、BeautifulSoup
技术指纹识别	基于 2850 余条指纹规则识别网站技术栈，输出置信度与命中证据	面向对象（多态）
漏洞主动验证	按指纹结果路由 2613 条社区模板与 42 条自研无害检测项，命中即「已验证漏洞」	面向对象（继承）、文件解析
漏洞情报关联	识别结果关联 11024 条情报，按 OSV 版本区间输出三级判定，KEV 红标	数据库、组合
网络层检测	TLS 证书过期 / 自签 / 弱协议检测，SPF/DMARC/MX 邮件安全审计	socket、DNS 查询
源码审计与登录爆破	16 规则静态审计与 Python 污点分析；授权前提下的登录表单爆破测试	AST 解析、HTTP 会话
工作台与数据管理	Web 工作台、批量扫描、扫描历史与 diff、四格式导出、REST API	Flask、PostgreSQL、进度显示

核心链路上，被动指纹识别、漏洞主动验证与漏洞情报关联三个模块构成递进关系：指纹识别回答「目标是什么」，主动验证回答「疑似漏洞是否真实存在」，情报关联回答「这个漏洞对目标是否构成实际风险」。三者缺一不可——只有指纹会沦为「同名就报漏洞」的误报机器，只有模板验证会缺乏优先级与上下文，只有情报则成了纸上谈兵。

外围模块面向资产管理与授权测试的日常使用：批量扫描支持一次提交最多 50 个站点并实时展示进度；扫描历史支持任意两次扫描的 diff 对比（新增 / 消失 / 版本变化），用于持续监控技术栈变化；情报库提供全库模糊与语义检索；导出覆盖

JSON、明细 CSV、78 列宽表 CSV 与单文件 HTML 报告四种格式，满足入库、报表与归档的不同需要；REST API 全量开放系统能力，便于接入自动化流水线。

以一个典型使用场景说明功能需求：安全人员输入授权目标网址，选择「深度识别」等级，系统依次完成目标校验、证据采集、指纹识别、模板验证与情报关联，页面实时显示当前阶段与进度；扫描完成后给出技术清单（含置信度）、已验证漏洞、情报三级判定、安全响应头评分与修复建议；人员可将结果导出为宽表 CSV 归档，或留存至历史供下次对比。

进一步地，把核心功能拆解为用户故事以明确验收口径：作为安全测试人员，我希望输入网址后 15 秒内看到目标的技术栈清单，以便快速判断资产构成；我希望每条识别结果都附带「它是如何被识别出来的」证据，以便向客户解释；我希望系统只提示「确认受影响」的漏洞，而不是把同名组件的所有历史漏洞倾倒给我；我希望对登录后的页面也能扫描，而不是只看到登录页的壳；我希望两次扫描之间的一切变化被自动记录，以便持续跟踪资产漂移。上述故事分别对应指纹识别、证据展示、三级判定、认证扫描与历史 diff 五项能力，构成了后续设计与测试的主线。

目标安全校验模块的需求来自一个真实痛点：扫描工具一旦允许内网地址，就会被攻击者当成从外网探测内网的跳板（SSRF）。因此本模块的验收标准是刚性的——任何指向环回、私有、保留地址的输入必须被拒绝，且拒绝发生在发起任何网络请求之前。

数据采集与解析模块的需求是「一次采集、多路使用」：同一页面的 HTML 只抓取与解析一次，就要产出响应头、Cookie、meta、脚本、DOM 选择器、CSS 类名等全部证据，供七个检测器共享。这决定了证据对象必须在解析期一次性构建完整，而不是各检测器各自为战地重复请求页面。

技术指纹识别模块的核心需求是「可信」：每条识别结论必须能回答「依据什么证据」「有多大把握」两个问题。因此需求规定识别结果必须携带置信度与命中的原始证据，短词匹配必须加边界以防「plex 命中 complex」式的荒谬结论。

漏洞主动验证模块的需求边界最为严格：只允许只读无害检测、命中必须二次确认、检测项必须与已识别技术相关联（避免无意义空跑）。这既是合规要求，也是误报控制的工程手段。

漏洞情报关联模块要求「提示有据、排除有理」：确认受影响的结论必须给出版

本区间依据，排除的结论必须能说明版本落在哪个区间之外；在野利用清单命中必须显著标注，帮助使用者排定处置优先级。

网络层检测模块把视角从 HTTP 扩展到传输层与域名体系，需求强调「轻触」：端口探测每端口至多两次交互，邮件安全检测只做记录查询不做邮件发送，全部检测项默认关闭。

源码审计与登录爆破模块面向两类补充场景：前者让开发者在上线前自查代码中的危险模式，后者在获得书面授权后评估登录接口的口令强度。两者的共性是风险较高，因此都必须置于明确的使用限制之下。

工作台与数据管理模块是全部能力的统一入口，需求强调「每个功能都有页面承载、每个结论都能展开证据」：批量、历史、检索、导出、API 文档缺一不可，避免出现有能力但找不到入口的情况。



图 2-1 系统首页（输入入口与实时统计）

2.2 性能需求分析

性能需求围绕「合规优先、够用高效」制定，各项指标如表 2-2 所示。

表 2-2 性能需求与实测结果

指标	需求	实测结果
----	----	------

指标	需求	实测结果
访问礼貌性	全局请求间隔 $\geq 0.4s$, UA 自报身份, 遵循 robots.txt	达标 (RateLimiter 全局生效)
并发能力	同时扫描任务 ≤ 3 , 超出排队并实时显示进度	达标 (信号量 + jobs 队列)
快速扫描耗时	单站 $\leq 15s$	实测约 5.7s (演示站点标准识别)
全面扫描耗时	含模板验证 $\leq 90s$	实测约 59s (演示站点)
指纹匹配正确性	短词不误报 (如 plex 不得命中 complex)	达标 (非字母数字边界匹配)
验证误报控制	模板命中须二次确认, 软 404 不误报	达标 (基线比对 + 重放确认)
数据安全	SQL 全参数绑定, 上传源码审计后即删	达标 (静态安全审计通过)

其中访问礼貌性是安全工具的第一约束: 扫描器对目标而言是一个「高频率的陌生访客」, 如果不加约束, 极易对小型站点造成事实上的压力测试, 也会触发防护设备封禁。因此系统把限速做成了全局强制项而非可选配置, 任何模块发起的请求都必须经过同一个限速器。数据安全方面, 系统全部数据库访问采用字面量 SQL 加 %s 参数绑定, 从编码层面杜绝拼接注入; 源码审计功能接收的用户上传内容在临时目录中处理, 审计结束立即删除, 不在服务器留下任何副本。

2.3 运行环境与约束

系统的目标运行环境与部署约束如表 2-3 所示。整体原则是「普通电脑即可运行、不依赖任何云服务」: 全部组件 (Web 服务、扫描引擎、数据库) 都运行在本机, 网络方面仅要求能够访问被授权扫描的目标站点; 漏洞情报的自动更新为可选功能, 在内网或离线环境中可直接关闭, 不影响扫描功能本身。

表 2-3 运行环境与约束

类别	要求	说明
操作系统	Windows 10/11、Linux 均可	开发与测试在 Windows 下完成
运行时	Python 3.10 及以上	依赖 Flask 3、requests、BeautifulSoup4、psycopg2、PyYAML
数据库	PostgreSQL 12 及以上 (实测 18)	本地实例, 库名 sitelens, 首次启动自动建表播种

类别	要求	说明
硬件	4 核 CPU / 8GB 内存 / 2GB 磁盘	知识库入库峰值占用约 1.5GB 磁盘
网络	可访问授权目标站点	出站 HTTP/HTTPS 与 DNS 查询; 无任何入站要求
合规约束	仅扫描已授权目标	内网目标被校验器拒绝; 主动能力默认关闭

3 总体设计

本章从系统分层、面向对象结构与数据库三个方面说明总体设计。

3.1 系统总体设计

系统采用四层架构,如图 3-1 所示。展示层与应用层之间以 fetch/JSON 交互,应用层与核心层之间通过异步任务队列解耦:扫描请求入队后由后台线程执行,前端轮询任务进度,避免了长请求阻塞。核心层按职责划分为八个高内聚模块,全部通过显式接口与数据类交互,不共享可变全局状态。

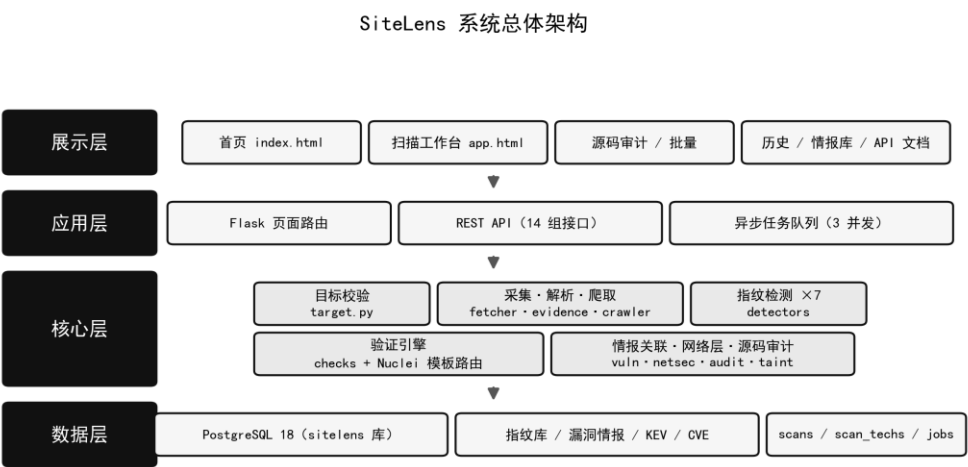


图 3-1 系统总体架构

表 3-1 系统分层职责

层次	组成	职责
展示层	index / app / audit / batch / history / intel / api-docs / about 八页	输入交互、进度渲染、结果可视化，零构建无框架
应用层	app.py: 页面路由 + REST API + 任务队列	请求调度、参数校验、异步执行、Token 鉴权
核心层	scanner 包: target / fetcher / evidence / crawler / detectors / checks / vuln / netsec / audit / taint	扫描链路全部业务逻辑，纯库无界面
数据层	PostgreSQL 18 (sitelens 库) 9 张表	知识库与扫描结果持久化，支持检索与统计

面向对象结构的核心是 ScannerEngine 组合根与 BaseDetector 继承体系，如图 3-2 所示。引擎在构造时持有采集器、爬虫、指纹库索引与情报匹配器四个组件（组合关系），扫描时对检测器列表统一调用 detect() 接口（多态），不区分具体类型；新增检测器只需继承基类并注册一行，引擎零改动。实体类（ScanTarget、PageEvidence、ScanResult、SecurityReport 等）全部采用双下划线私有属性加只读 property 对外暴露，仅提供受控写入点——例如 PageEvidence 仅允许解析器回填一次标题，Technology 的置信度在新增独立证据来源时按规则自动提升——保证扫描过程中的数据不可被外部随意篡改。四大面向对象特性的落点汇总见表 3-2。

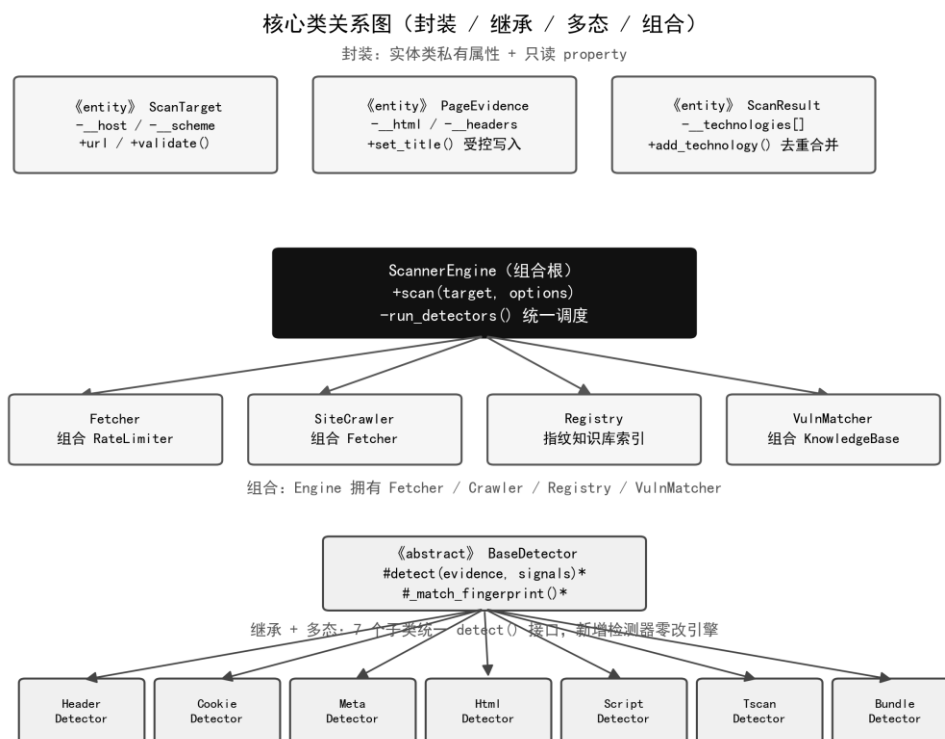


图 3-2 核心类关系图

表 3-2 面向对象四大特性的实现落点

面向对象特性	实现位置	设计要点
封装	models.py 全部实体类	双下划线私有属性 + 只读 property；仅 set_title()、add_evidence() 两个受控写入点
继承	detectors/base.py → 7 个子类	BaseDetector 抽象模板方法 detect()，子类各消费一类证据
多态	engine._run_detectors()	对检测器列表统一调用接口，新增检测器引擎零改动
组合	ScannerEngine 及各组件	Engine 拥有 Fetcher/Crawler/Registry/VulnMatcher；Fetcher 内组合 RateLimiter

这种结构带来的直接收益是扩展成本极低：实训后期相继引入表达式指纹检测器、bundle 签名检测器与验证引擎，均未修改引擎主流程；数据驱动的指纹规则（数据仓库而非硬编码）则让指纹库扩充完全脱离代码发布。

总体设计过程中有四个关键取舍，记录如表 3-6 所示——它们决定了系统后续所有迭代的走向，也是答辩中最常被问到的设计问题。

表 3-6 关键设计决策与权衡

设计决策	备选方案	选择理由
指纹规则存数据库 (JSONB)	硬编码为 Python 文件	规则增删不改代码；导入管线直接写库；检索走索引
扫描结果 JSONB 主表 + 关系型明细表	只用关系型明细表	JSONB 保证结果完备可回溯；明细表保证反查走索引
限速器全局唯一	每个模块各自限速	多模块并发时限速才真实有效，避免叠加超速
前端零构建	Vue/React + 打包链	实训交付零工具链依赖；页面数量少，手写成本可控

3.2 数据库设计

系统使用 PostgreSQL 18 存储知识库与扫描数据，数据库名为 sitelens，共设计九张表，如表 3-3 所示。知识库表与结果表分离：categories、technologies、tscan_fingerprints、service_fp、vuln_kb、cve_ms、kev 为静态或定期更新的知识数据；scans、scan_techs、jobs 为扫描产生的业务数据。

表 3-3 数据库主要数据表

表名	用途	规模（实测）
categories	78 个检测类别字典(ord 列即宽表 CSV 列序)	78 条
technologies	自建精编指纹 (rules JSONB : headers/cookies/meta/html/dom/scripts)	370 条
tscan_fingerprints	社区表达式指纹 (groups JSONB: 组内 AND、组间 OR)	2481 条
service_fp	nmap 风格端口服务指纹(pattern 预验证可编译)	11966 条
vuln_kb	漏洞情报库 (product trgm 索引 + affected 版本区间 + 256 维语义向量)	11024 条
cve_ms	微软安全公告索引 (component 索引)	34931 行
kev	CISA 在野利用清单	1695 条
scans / scan_techs	扫描主表 (JSONB 全量) + 技术明细 (关系型可检索)	随使用增长
jobs	异步扫描/爆破任务队列 (状态、进度、结果)	随使用增长

以核心的漏洞情报表 vuln_kb 为例，其字段设计如表 3-4 所示。product 字段建立 trgm 模糊索引 (GIN) 支持亚秒级相似检索；affected 字段以「|」分隔存储多组 OSV 版本区间；embedding 字段存储 256 维哈希 TF-IDF 向量用于语义排序。扫描结果主表 scans 以 JSONB 保存全量结果保证完备性，明细表 scan_techs 抽出

「站点-技术」关系支持反向检索。

表 3-4 vuln_kb 漏洞情报表主要字段

字段	类型	说明
id / cve	TEXT	情报唯一标识与关联 CVE 编号
product / name	TEXT	受影响产品名 (trgm 索引) 与漏洞名称
severity	TEXT	critical / high / medium / low / info
affected	TEXT	OSV 版本区间, 多组以「 」分隔, 如 >=8.3, <8.3.7 2.x
cvss_v3 / embedding	TEXT / float8[]	CVSS 评分与 256 维语义向量
src / ref	TEXT	来源 (afrog/xray/tscan/ghsa/osv) 与参考链接

扫描结果主表 scans 与技术明细表 scan_techs 的设计兼顾「存得全」与「查得快」, 如表 3-5 所示: 主表以 JSONB 保存单次扫描的全部原始结果, 保证任何结论都可回溯; 明细表把「站点-技术」关系抽成关系型行, 使「哪些站点使用了某技术」这类反向检索可以走索引完成。

表 3-5 扫描结果表主要字段

表. 字段	类型	说明
scans.id / url / title	BIGINT / TEXT	扫描记录标识、目标地址与页面标题
scans.result	JSONB	全量扫描结果 (技术、证据、评分、情报), 单条可达数百 KB
scans.headers_score	TEXT / INT	安全响应头等级 (A+ - F) 与加权得分
scans.created_at	TIMESTAMPTZ	扫描时间, 支持历史排序与 diff
scan_techs.scan_id	BIGINT	外键关联扫描记录
scan_techs.tech_name / version / confidence	TEXT / TEXT / INT	技术名、版本与置信度, 建索引支持按技术反查站点

全部数据库访问统一封装在 storage 层, 采用字面量 SQL 加 %s 参数绑定, 静态安全扫描通过; 模糊检索使用 position()/similarity() 函数而非字符串拼接, 动态排序字段以白名单映射, 不存在任何注入面。

3.3 界面设计原则

前端遵循「内容优先、克制装饰」的设计原则: 全局使用一套实测得来的设计令

牌（背景、文字、边框、圆角、硬阴影）驱动明暗双主题，所有组件只消费令牌而不各自定义颜色，保证了视觉的一致性；标题与数据采用等宽字体以呼应安全工具的终端气质，正文使用适合长时间阅读的无衬线字体。

可读性与可访问性同样被纳入设计约束：全部交互元素提供键盘焦点环与悬停反馈，动效尊重系统的「减少动态效果」偏好，长列表提供骨架屏与空态提示，滚动条与选中色随主题联动。响应式布局在窄屏下自动收起侧栏、堆叠内容区，保证笔记本与投影环境下的可用性。

4 详细设计与实现

本章按扫描链路顺序展开各模块的实现。系统一次完整扫描的主流程如图 4-1 所示，共八个阶段，全部阶段进度实时上报前端。

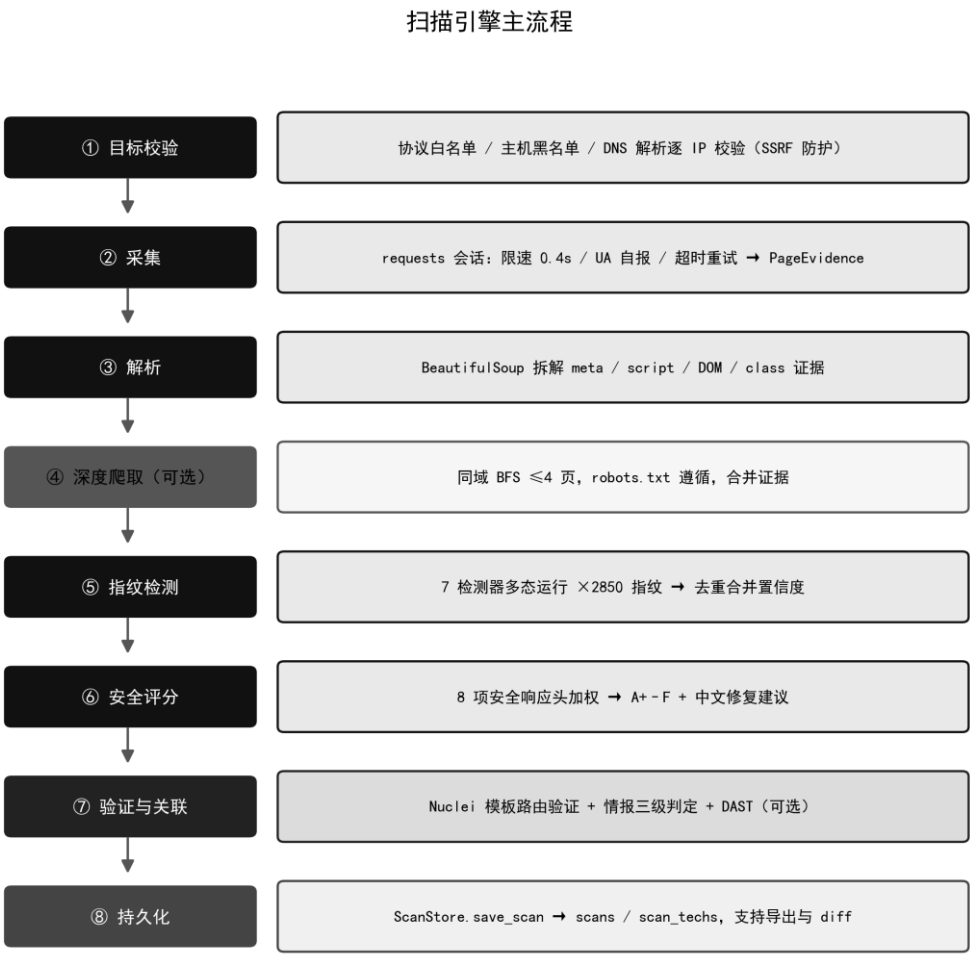


图 4-1 扫描引擎主流程

4.1 目标校验与扫描引擎主流程

目标校验是扫描前的安全闸门。ScanTarget 类对用户输入依次执行三项检查：协议白名单（仅允许 http/https）、主机名黑名单（拒绝 localhost、*.local、*.internal 与云元数据地址）、DNS 解析后逐 IP 校验（拒绝环回、私有、链路本地、保留与组播地址）。逐 IP 校验使得借助 DNS 重绑定绕过黑名单的攻击方式失效——即便域名首次解析返回公网地址，扫描器仍会在每次请求前校验实际连接的 IP，工具自身不会被利用为访问内网的跳板。校验不通过时抛出 TargetError，由应用层转换为明确的中文错误提示返回前端。

主流程由 ScannerEngine.scan() 编排，各阶段通过 progress 回调实时上报「已完成/总数 + 当前动作」，前端据此渲染进度条与阶段面板；任一阶段抛出的异常都被捕获并转换为任务错误状态，不会导致服务崩溃。扫描过程中产生的所有中间数据（证据对象、命中记录、评分）都以只读视图暴露给上层，保证结果不可被扫描流程之外的代码篡改。

目标校验器的核心判定逻辑如下——对 DNS 解析得到的每一个 IP 都做地址类别校验，任何一项不满足即整体拒绝：

```
def validate(host, resolved_ips):
    if scheme not in ("http", "https"):          # ① 协议白名单
        raise TargetError("仅支持 http/https 目标")
    if blacklisted(host):                        # ② 主机黑名单
        raise TargetError("目标主机不被允许")
    for ip in resolved_ips:                      # ③ 逐 IP 地址校验
        if loopback(ip) or private(ip) or reserved(ip):
            raise TargetError("目标解析到受限地址")
```

八个阶段的职责与关键实现点汇总如表 4-1 所示。

表 4-1 扫描八阶段职责一览

阶段	职责	关键实现点
① 目标校验	过滤非法与危险目标	协议白名单；DNS 解析逐 IP 校验，防 DNS 重绑定
② 采集	获取页面原始数据	requests 会话；0.4s 全局限速；超时重试；UA 自报
③ 解析	HTML 转结构化证据	BeautifulSoup 单次遍历产出多类信号
④ 深度爬取	扩大证据覆盖面	同域 BFS ≤4 页；robots.txt 遵循；证据合并

阶段	职责	关键实现点
⑤ 指纹检测	识别技术栈	7 检测器多态运行；置信度合并；版本捕获
⑥ 安全评分	评估响应头配置	8 项安全头加权；A+ - F 等级；中文修复建议
⑦ 验证与关联	证实漏洞与风险	模板路由验证；情报三级判定；DAST（可选）
⑧ 持久化	结果落库与导出	JSONB 主表 + 明细表；任务状态回写

4.2 数据采集与解析模块

采集模块 `Fetcher` 封装 `requests.Session`，核心是「全局唯一」的限速器：无论哪个模块发起请求，都必须经过同一个 `RateLimiter`，保证对任意目标的请求间隔不低于 0.4 秒。其实现只在进程内维护上次请求时间戳，每次请求前计算需要等待的剩余时间：

```
class RateLimiter:
    def __init__(self, interval):
        self.__interval = interval      # 最小请求间隔（秒）
        self.__last = 0.0              # 上次请求时间戳

    def wait(self):
        elapsed = time.time() - self.__last
        if elapsed < self.__interval:
            time.sleep(self.__interval - elapsed)
        self.__last = time.time()
```

解析模块基于 `BeautifulSoup` 将 HTML 拆解为结构化证据对象 `PageEvidence`，包括 `meta` 标签、脚本地址与内联脚本、样式表、CSS 类名集合、DOM 选择器命中池与站内链接清单；`favicon` 图标会计算 `mmh3` 哈希用于 CDN 识别。深度模式下爬虫 `SiteCrawler` 以目标 URL 为起点做同域广度优先爬取，最多 4 页，爬取前先解析 `robots.txt` 并跳过 `Disallow` 路径；多页证据合并后送入检测器，显著提升了仅在子页面露出的组件（如统计脚本、客服组件）的检出率。

证据解析的核心是「一次解析、多路复用」：`BeautifulSoup` 只遍历一次 DOM，同时产出多类结构化信号供七个检测器共享，避免重复解析带来的性能浪费。关键实现如下：

```
def extract_signals(html):
    soup = BeautifulSoup(html, "html.parser")
    signals = {
        "meta": {m.get("name").lower(): m.get("content")
                  for m in soup.find_all("meta") if m.get("name")},
```

```

"scripts": [s.get("src") for s in soup.find_all("script")
             if s.get("src")],
"css_classes": collect_classes(soup),    # CSS 类名集合
"links": collect_same_domain_links(soup),
}
return signals                          # 供 7 个检测器共享

```

扫描工作台界面如图 4-2 所示：左侧为扫描配置（目标、等级、认证会话、可选 UA），右侧主区在空闲态给出引导，扫描开始后切换为实时阶段面板与进度条。



图 4-2 扫描工作台界面

4.3 技术指纹识别模块

指纹识别采用「证据驱动 + 规则数据化」的设计。指纹规则全部存于数据库：370 条自建精编指纹以 JSONB 描述 headers/cookies/meta/html/dom/scripts 六类匹配条件；2481 条社区指纹则以表达式组存储（组内 AND、组间 OR，支持 != 取反），导入期预解析为结构化组，运行期仅做子串与边界匹配，单页全量匹配耗时低于 1 秒。

一条自建精编指纹的存储结构示例如下——同一指纹可同时声明多类匹配条件，全部满足才判定命中，从规则层面抑制误报：

```

{
  "name": "Vite",
  "category": "构建工具",
  "rules": {

```

```

    "headers": {"x-powered-by": "vite"},
    "html":     ["/\\@vite/client"],
    "scripts":  ["/node_modules/.vite/"],
    "dom":      ["vite-error-overlay"]
  }
}

```

七个检测器按证据类型分工：Header、Cookie、Meta、Html、Script 五个检测器分别消费对应证据字段，TscanDetector 独立解释表达式组，BundleDetector 抓取同域主 bundle 文件做库签名匹配，补齐「打包进产物后消失」的前端框架。识别结果经 add_technology() 去重合并：同一技术的多个独立证据来源会合并为一条记录，置信度随来源数提升（每来源 +5，上限 100），版本号从响应头、generator 标签与脚本文件名中用正则捕获。

以对真实站点 演示站点 的实测为例：深度识别共检出 13 项技术，按类别分组展示（内容分发网络 Cloudflare、托管平台 Vercel、性能类 HTTP/3 与 Cloudflare Analytics、SEO 类 Open Graph / Twitter Cards / schema.org 等），每条均可展开查看命中证据；整站识别耗时约 5.7 秒，结果如图 4-3 所示。



图 4-3 扫描结果：按类别分组的技术指纹

4.4 验证型检测引擎

验证引擎回答「漏洞是否真的存在」。自研检测项共 42 条，全部为只读无害类

（如 .git 泄露、备份文件、Spring Actuator 暴露、目录列表开启、CORS 反射等），并额外适配了 2613 条 Nuclei 社区模板。模板调度采用 MoE 式三路路由（图 4-2）：指纹 tag 硬匹配置顶、语义向量余弦排序、轮转游标兜底，每轮仅激活 80 条模板，避免无差别空跑。

Nuclei 模板并非直接运行，而是经过一个「转换器」预处理的：官方模板库的 http 目录共收录上万个 yaml 模板，转换器只保留能够静态翻译为本系统检测格式的子集——单 GET 请求、路径仅为站点相对路径、匹配器为状态码与关键词两类；其余模板因依赖表达式求值、多请求编排、模糊测试载荷或外部回调服务器而被跳过。转换漏斗如表 4-2 所示，这一数字是我们对「兼容生态」最诚实的度量。

表 4-2 Nuclei 模板转换漏斗

被跳过的模板特征	数量（约）	对应缺失的引擎能力
使用 dsl 表达式匹配器	3500+	表达式求值引擎
使用 flow 多请求编排	1100+	多步骤请求状态机
使用 payloads 模糊测试	690+	注入与迭代引擎
依赖 interactsh 外部回调	540+	OOB 回连服务器
转换成功入库	2613	单 GET + 关键词匹配

漏斗同时指明了扩容路线：每补齐一项引擎能力即可解锁对应类别的模板，其中 POST 方法支持与简单比较表达式成本最低，OOB 回连成本最高（见总结章的改进计划）。

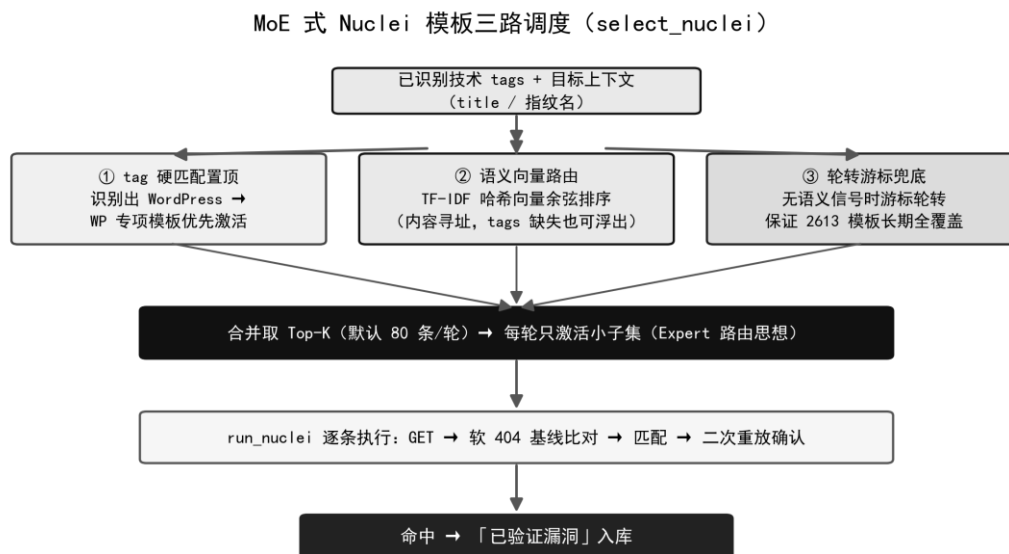


图 4-4 MoE 式 Nuclei 模板三路调度

三路调度的核心实现如下：tag 命中的模板直接进入优先队列；其余模板与目标上下文（标题 + 指纹名）计算余弦相似度排序；无语义信号时按文件游标轮转，保证模板库长期全覆盖。

```

def select_nuclei(cap=80, tech_tags=(), query_text=""):
    rows, vecs = load_nuclei(), _nuclei_vectors(load_nuclei())
    tags = {t.lower().strip() for t in tech_tags if t}
    hit, rest = [], []
    qvec = embed_text(query_text) if query_text else None
    for r, vec in zip(rows, vecs):
        if tags & {t.lower() for t in r.get("tags", [])}:
            hit.append(r)  # ① 指纹 tag 置顶
        else:
            sim = cosine(qvec, vec) if qvec else 0.0
            rest.append((_SEV_ORDER[r["sev"]] - sim * 10, r))
    rest.sort(key=lambda x: x[0])  # ② 语义相似度排序
    rest = rotate_by_cursor(rest, cap)  # ③ 游标轮转兜底
    return (hit + rest)[:cap]
  
```

执行阶段每条模板只允许请求站内相对路径（防止伪造目标），先请求不存在的随机路径建立软 404 基线，与基线相同的响应直接跳过；命中后再立即重放一次，两次均命中才采信，以排除 CDN 多节点响应抖动造成的假阳性。命中记录统一升级为「已验证漏洞」，与情报提示分区呈现。匹配判定逻辑如下：

```

def _group_match(group, status, body_l, header_l):
    if group.get("s") and status not in group["s"]:  # 状态码
        return False
    for w in group.get("wall", []):  # 组内 AND 关键词
        if w not in body_l: return False
  
```

```
if group.get("wany") and not any(w in body_1      # 组间 OR 关键词
    for w in group["wany"]): return False
for n in group.get("n", []):                      # 排除词（防误报）
    if n in body_1: return False
return True
```

此外引擎还包含轻量 DAST 检测（反射 XSS 标记回显、报错型 SQL 注入、开放重定向、目录遍历）与认证扫描能力：用户可注入 Cookie 会话使爬取与检测覆盖登录后才可见的页面。

扫描结果以结构化 JSON 落库与导出，已验证漏洞条目的形态如下——检测项、严重度、命中地址与修复建议一一对应：

```
{
  "check": "web-git-leak",
  "title": "Git 仓库泄露",
  "severity": "high",
  "url": "https://target/.git/HEAD",
  "evidence": "HTTP 200（二次确认）",
  "advice": "删除 Web 根下 .git 目录，部署流程加导出过滤",
  "src": "self"
}
```

轻量 DAST 的四项检测均基于「注入安全标记 → 观察响应」的只读思路实现，如表 4-2 所示——不使用任何破坏性载荷，参数级探测上限 15 个。

表 4-2 轻量 DAST 检测项

检测项	探测方式	判定特征
反射型 XSS	注入唯一标记并检查回显上下文	标记未转义出现在响应体中
报错型 SQL 注入	注入单个引号触发数据库报错	响应包含典型 SQL 错误特征串
开放重定向	注入外部地址参数	响应 3xx 且跳转指向注入地址
目录遍历	逐层回溯探测固定文件	响应包含预期文件特征内容

认证扫描面向「登录后才可见」的攻击面：用户在工作台粘贴已登录会话的 Cookie，采集与检测链路会携带该会话执行，从而覆盖认证后页面；系统同时会在结果中标注哪些发现来自认证后的页面，便于与匿名视角的扫描结果对照。

4.5 漏洞情报关联模块

情报模块 VulnMatcher 将识别结果映射到已知漏洞，三级判定流程如图 4-5。系统内置 11024 条漏洞情报（afrog/xray/TscanPlus 的 POC 元数据，载荷不落盘不执行）、1323 条带 OSV 版本区间的结构化记录与 1695 条 CISA KEV 在野利用清单。判定规则：当目标组件带精确版本号且存在区间数据时，仅输出「确认受影响」

结论：同名但无版本号输出「可能受影响」提示；版本落在区间之外直接过滤。命中 KEV 清单的情报附加红标，提示该漏洞已被在野利用、应优先处置。

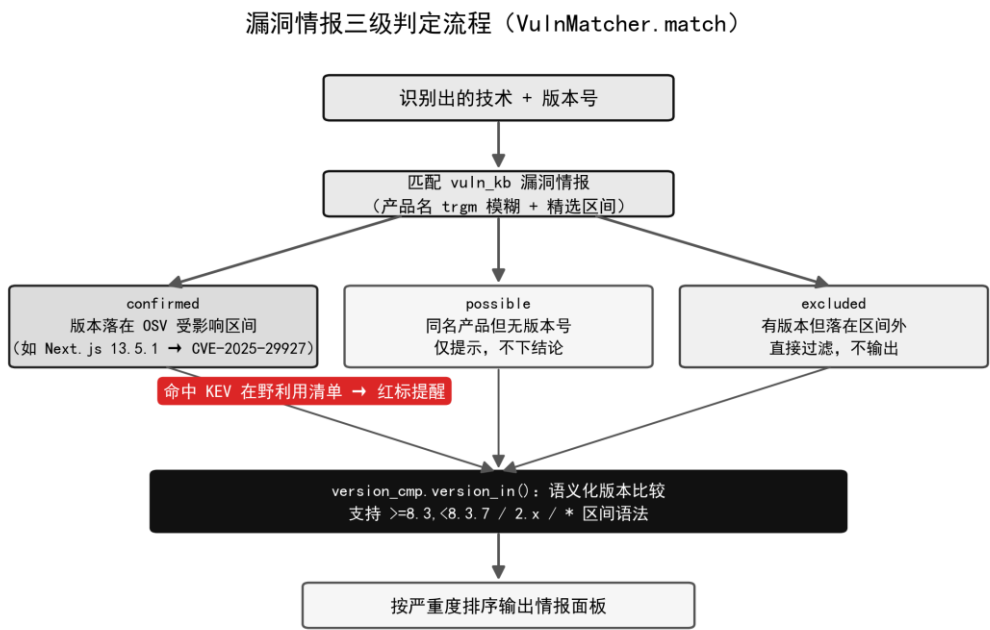


图 4-5 漏洞情报三级判定流程

版本比较器支持 ≥ 8.3 , $< 8.3.7$ 、 $2.x$ 、 $*$ 等区间语法，可表达 OSV 数据的全部区间形态；情报检索采用 trgm 相似度粗筛(GIN 索引)加向量余弦重排的混合方案。情报库检索页支持按产品名、关键词实时检索，如图 4-6 所示。

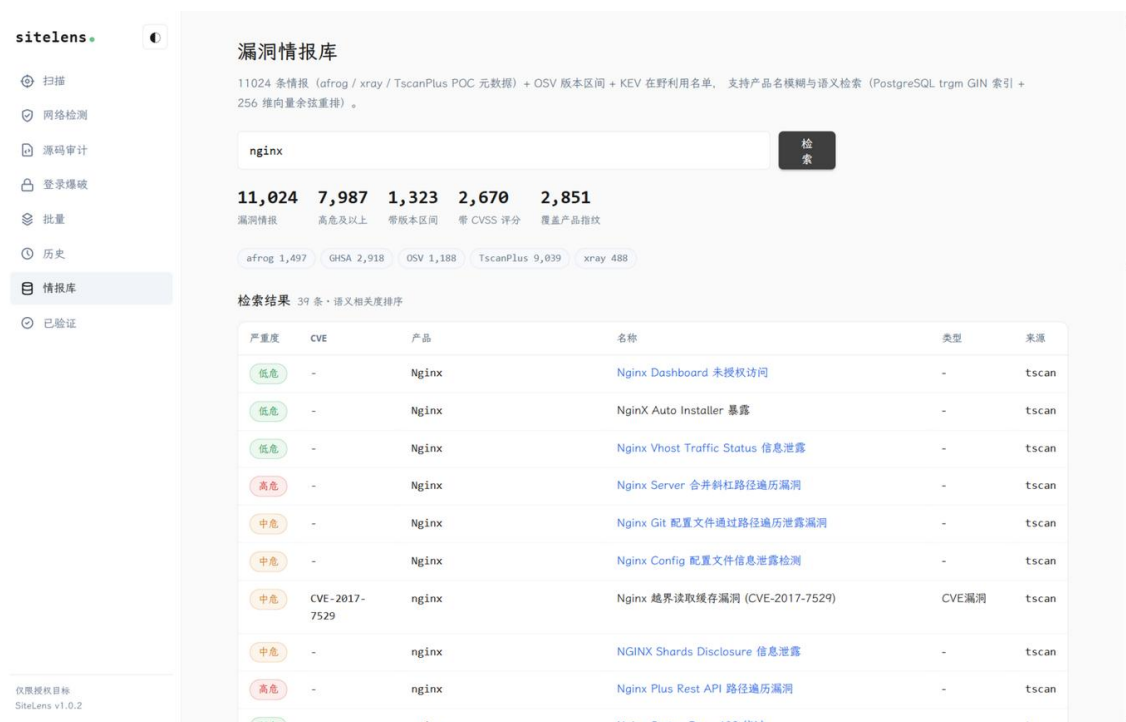


图 4-6 漏洞情报库检索

情报的时效性由自动更新管线保障：启动时守护线程先补一次增量，此后每 24 小时拉取一次 OSV 版本区间与 CISA KEV 在野利用清单，解析后以事务写入 vuln_kb 与 kev 表（可通过环境变量关闭）。更新管线与扫描引擎完全解耦——情报行数变化会自动触发检索侧的向量缓存重建，无需重启服务。截至测试结束，带区间情报已由初始的 909 条增长到 1323 条，验证了管线的实际运转。

以一次真实扫描的三级判定走查作为本节小结：演示站点被识别为 Next.js 且版本捕获为 13.5 系列，情报匹配命中 CVE-2025-29927（Next.js Middleware 越权绕过），其 OSV 区间覆盖 13.5.x，版本落入区间 → 输出「确认受影响」，严重度「严重」，来源标注 afrog 与 tscan 双源；同时检出的部分历史漏洞因目标版本不在受影响区间而被自动排除，不再出现在结论中。整个过程无需人工介入，误报被机制性地压制在判定层。

4.6 网络层检测模块

网络层检测从传输层与域名体系两个视角补充 Web 层看不到的问题。TLS 检测项包括：证书链是否过期或 30 天内临期、是否自签名、证书域名与目标是否匹配、是否仍协商 SSLv3/TLS1.0 等旧协议；邮件安全检测通过 nslookup 查询 SPF、DMARC、

MX 记录，判断域名是否可被伪造发件，检测前先将子域名归位到组织主域（如 send.example.com 的 SPF 问题应归并到 example.com 评估），避免误报；端口服务识别对 6 个常见 Web 端口做 TCP 连接，先被动读取 banner，静默则发送最小 HTTP 探测，用 11966 条 nmap 风格指纹匹配服务与版本。全部检测均为只读操作，默认关闭、显式开启。界面如图 4-7 所示。

表 4-1 网络层检测项（节选）

检测项	判定条件	结论
证书过期	notAfter 早于当前时间	高危：连接可被中间人劫持
证书临期	notAfter 距今不足 30 天	提醒：尽快更换证书
自签名证书	颁发者与主体相同且非受信 CA	高危：身份无法被浏览器验证
域名不匹配	目标域名不在证书 SAN 列表	高危：证书与站点不符
弱协议协商	可完成 SSLv3 / TLS1.0 握手	中危：应禁用旧版协议
邮件伪造风险	缺少 SPF/DMARC 或策略为 p=none	中危：第三方可伪造本域发件

邮件安全检测的解析环节值得一提：中文 Windows 系统下 nslookup 的输出为 GBK 编码，且 TXT 记录值分散在「text =」续行的引号内，直接按行读取会得到空结果——这正是早期「多个域名检测结果雷同」缺陷的根因。修复后的解析器按记录段收集全部续行并剥离引号，再把子域名的检测结果归位到组织主域统一评估。

端口服务识别的实现刻意保持「轻触」风格：对 80、443、8080 等 6 个常见 Web 端口发起连接后，先静默等待 2 秒读取服务主动推送的 banner，仅在无响应时发送一个最小化的 HTTP 探测；取得的指纹文本与 11966 条 nmap 风格特征做匹配，输出服务名称与版本。整个过程每个端口至多两次交互，且可在配置中关闭。



图 4-7 网络层检测界面

4.7 登录爆破与源码审计模块

登录爆破面向授权测试场景：先用 BeautifulSoup 解析登录页表单（兼容无 form 标签、字段无 name、中文 placeholder 等情况），自动识别验证码字段，随后按用户名-密码字典组合提交，以失败基线加响应差异判定命中，总请求数硬性封顶 300 次；每次尝试前重新加载登录页以刷新验证码，配合本地 OCR 实现数字/运算/点选三类验证码的分级处理。模块入口设置了授权确认门禁，未勾选授权声明无法启动。

验证码分级处理框架如表 4-3 所示，全部推理在本地完成，不依赖任何外部打码服务。

表 4-3 验证码分级处理

验证码类型	识别方式	边界说明
数字/字母算式	本地 OCR + 四则运算求值	常规算式验证码可自动处理
点选式	检测框坐标定位 + 点击坐标回传	依赖目标图片质量
滑块式	缺口检测给出偏移量	真实拖动轨迹需浏览器自动化，仅提供偏移

源码审计模块支持上传源码文件或压缩包，16 条危险规则覆盖命令执行、反序列化、任意文件读取等模式；污点分析基于 ast 模块追踪「不可信输入到危险执行

点」的同函数赋值链——把 request.args 等用户输入标记为 source，把危险执行函数标记为 sink，赋值与字符串拼接都会传播污点标记，比单行正则更接近真实数据流。审计在临时目录完成，结束后立即删除全部上传内容。界面如图 4-8 所示。

污点分析的数据流模型如下——source 集合覆盖 Web 请求的各类入口，sink 集合覆盖常见危险执行点，赋值与字符串格式化都会沿变量传播污点：

```
SOURCES = {"input", "request.args", "request.form",
           "request.cookies", "sys.argv"} # 不可信数据入口
SINKS    = {"危险执行函数集合",
            "(执行类 / 反序列化类 / 文件写入类)" } # 危险落点

# 判定逻辑: source → 赋值/拼接 → sink, 即报告高危数据流
name = request.args.get("name") # source: 不可信输入
msg  = "hi, " + name            # 污点随赋值与拼接传播
danger(msg)                     # 若命中 sink 集合 → 高危告警
```



图 4-8 源码静态审计界面

4.8 Web 工作台与数据持久化模块

Web 工作台采用零构建的手写 HTML/CSS/JS 实现，明暗双主题，包含首页、扫描工作台、源码审计、批量扫描、扫描历史、情报库检索、API 文档与产品介绍八个页面。扫描以异步任务执行（3 并发信号量），前端轮询 /api/job/<id> 渲染实时阶段面板；结果页按「概要—技术分组—已验证漏洞—情报提示—安全评分」分区展示，

每条结论可展开原始证据。

持久化由 ScanStore/JobStore 封装：扫描主表存 JSONB 全量结果，技术明细表建立关系型索引以支持「哪些站点使用了某技术」的检索；历史页列出全部扫描记录，勾选两次即可 diff 出新增、消失与版本变化的组件，点击任一记录可就地展开详情（概要、指纹、情报、评分），如图 4-9 所示；结果支持 JSON、明细 CSV、78 列宽表 CSV 与单文件 HTML 报告四种导出格式。对外 REST API 使用 Token 鉴权，全部接口在 API 文档页集中说明，如图 4-10 所示。

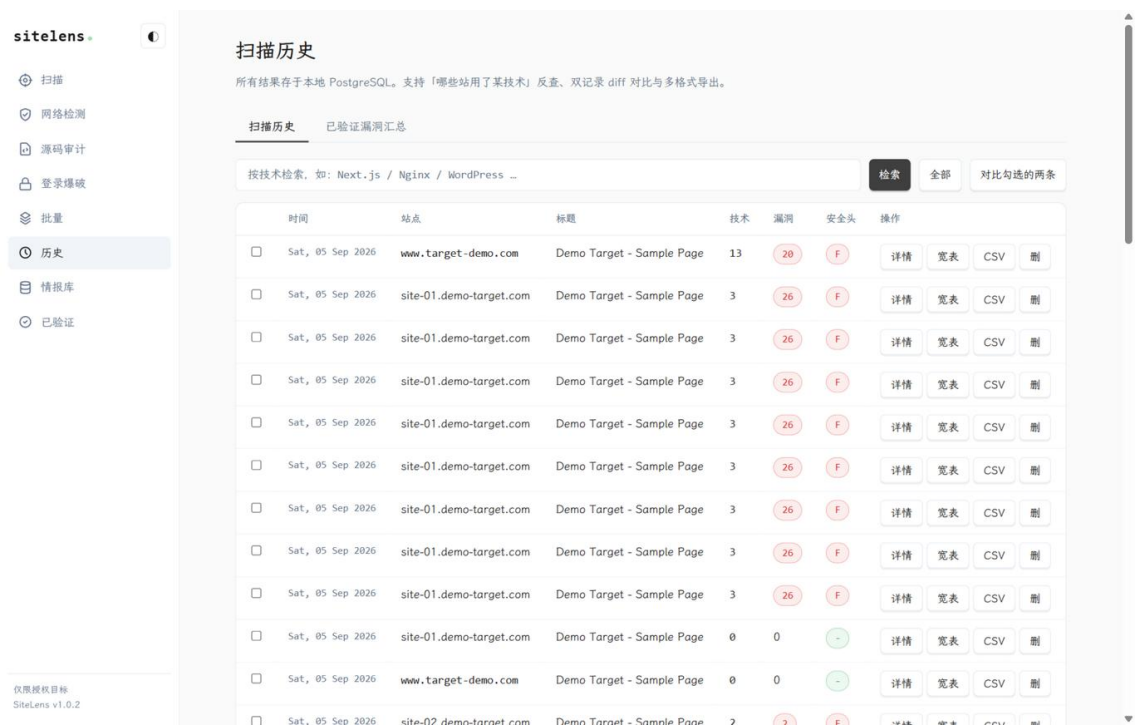


图 4-9 扫描历史与记录详情

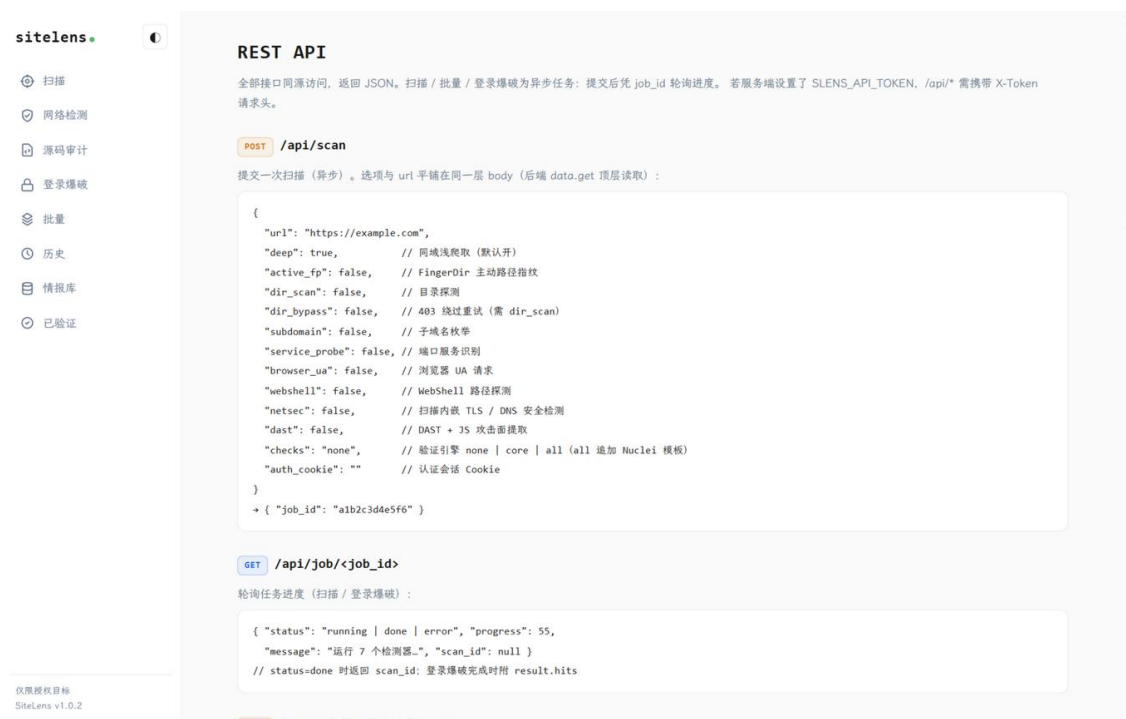


图 4-10 REST API 接口文档

报告导出覆盖四种格式，分别对应不同使用场景：JSON 保留全量结构化数据，用于程序化归档与二次分析；明细 CSV 按扫描记录逐行输出核心技术字段，适合表格软件查看；78 列宽表 CSV 以「每类检测一行」的宽表形态输出，可直接作为资产台账导入数据仓库；单文件 HTML 报告内联全部样式与数据，双击即可离线打开，适合作为交付物直接发给授权方。所有导出均由 Exporter 统一生成，数据源与页面展示完全一致。

工作台八个页面的功能定位汇总如表 4-4 所示——每个功能都有明确的页面承载，不存在「有能力但找不到入口」的情况。

表 4-4 Web 工作台页面清单

页面	定位	核心交互
首页 /	产品入口	输入网址直达扫描；实时统计；演示动画
扫描工作台 /app	核心操作台	等级选择、认证会话、实时阶段面板、结果分区
源码审计 /audit	代码安全 检查	拖拽上传、审计报告、样本演示
批量扫描 /batch	多站点任务	50 站点队列、逐条进度、汇总状态
扫描历史 /history	资产管理	记录详情、技术反查、diff 对比、四格式导出
情报库 /intel	知识检索	模糊 + 语义检索，严重度与来源标注
API 文档 /api-docs	集成入口	14 组接口说明与在线调试

页面	定位	核心交互
产品介绍 /about	对外说明	能力总览、数据来源署名、合规承诺

以「Cloudflare」的识别过程为例走查一次指纹命中：响应头集合中出现 cf-ray 与 server: cloudflare（Header 检测器命中），CSS 类名集合出现 cf-browser-verification（DOM 检测器命中），两个独立来源指向同一技术——去重合并为一条记录，置信度按规则提升，并在结果中同时保留两个证据供人工复核。这个最小的闭环展示了「证据驱动 + 多态检测器 + 受控合并」三个设计决策如何在真实数据上协同工作。

开放 API 的同时系统保持了严格的访问控制：受保护接口要求请求头携带与服务器配置一致的 Token，缺失或不匹配一律返回 401；结合 SSRF 校验与参数白名单，攻击者无法把本系统当作探测内网的跳板——扫描器的「武器化风险」在设计层面即被拆除。

4.9 系统部署与使用说明

系统部署遵循「三步可用」原则：安装依赖、配置数据库、启动服务，全部命令如下：

```
# ① 安装依赖（Python 3.10+）
pip install -r requirements.txt

# ② 配置数据库连接（项目根目录 .env）
# SLENS_DB_HOST / SLENS_DB_PORT / SLENS_DB_USER
# SLENS_DB_PASSWORD / SLENS_DB_NAME=sitelens

# ③ 启动：Web 工作台 / 命令行扫描
python app.py # Web 工作台，默认 5000 端口
python main.py scan <url> # 命令行快速扫描
python main.py scan <url> --full # 含主动模块，仅限授权目标
```

首次启动时系统自动建表并播种知识库（78 个检测类别与 370 条自建指纹）；可选执行资产导入脚本灌入社区指纹、漏洞情报与各类字典。部署环节最常见的三类问题与处置方法如表 4-5 所示。

表 4-5 部署常见问题与处置

常见问题	原因	处置方法
启动报数据库连接失败	数据库未启动或连接配置错误	检查 PostgreSQL 服务与 .env 配置项

常见问题	原因	处置方法
页面打开但扫描一直排队	并发任务已达 3 个上限	等待队列腾出，或调高信号量配置
情报检索无结果	知识库尚未完成导入	重跑资产导入脚本后重启服务

5 系统测试

本章从测试目的、测试环境与测试用例三个方面对系统进行验证。

5.1 测试目的

测试目的：（1）验证各功能模块在正常输入下输出正确结果，异常输入下优雅报错而非崩溃；（2）验证面向对象设计的可扩展性（新增检测器是否零改引擎）；（3）验证合规与安全约束真实生效（内网目标必须被拒绝、限速必须生效）；（4）评估真实站点的识别效果与误报率。

测试采用三层结构：第一层为 49 项单元/集成测试（unittest 框架，全部离线可重复，覆盖校验器、解析器、匹配器、比较器等核心逻辑）；第二层为 32 项 API 冒烟测试（覆盖页面可达、参数校验、边界条件、四种导出格式与统计接口）；第三层为真实环境实测（演示站点 等真实站点与自构造的漏洞样本页）。测试环境如表 5-1 所示。

测试的执行方式保证可重复性：单元测试以 unittest discover 一键运行，全部用例离线执行（不依赖外网与真实目标），构造的样本页与固定数据内置于仓库；冒烟测试脚本按「请求 → 状态码与结构断言」逐项校验接口契约；真实站点实测则在记录现场后与历史结果比对，确认识别能力的稳定性。

表 5-1 测试环境

项目	配置
操作系统	Windows 10/11 x64
处理器 / 内存	x64 四核及以上 / 16GB
Python 环境	Python 3.10 + Flask 3 + requests + BeautifulSoup4 + psycpg2
数据库	PostgreSQL 18 (sitelens 库，含全部知识库数据)
浏览器	Firefox / Edge 最新版 (前端验证)

项目	配置
测试靶标	演示站点（真实站）、自构造泄露样本页、内网地址（负向用例）

5.2 测试用例

主要功能测试用例及结果如表 5-2 所示。

表 5-2 主要功能测试用例

编号	测试项	测试方法	预期结果	结论
T01	目标校验	提交 http://127.0.0.1 与内网 IP	拒绝扫描并提示不合法目标	通过
T02	异常输入	提交空 URL、ftp:// 协议	返回明确错误信息	通过
T03	指纹识别	扫描演示站点（深度模式）	识别出 13+ 项技术并按类别分组	通过
T04	版本捕获	对带版本头的站点扫描	正确捕获如 nginx/1.24.0 版本号	通过
T05	置信度合并	多来源命中的同一组件	合并为一条且置信度提升	通过
T06	验证引擎	对构造的 .git 泄露靶页扫描	命中「Git 仓库泄露」已验证漏洞	通过
T07	软 404 基线	对不存在路径批量探测	自定义 404 页不被误报	通过
T08	二次确认	模拟间歇性异常响应	单次命中不采信，两次命中才上报	通过
T09	情报三级判定	构造 Next.js 13.5.1 站点	输出确认受影响 CVE-2025-29927	通过
T10	区间排除	构造已修复版本站点	excluded 结论不输出	通过
T11	KEV 红标	检索含 KEV 漏洞的组件	命中项带在野利用红标	通过
T12	语义检索	情报库检索 grafana	亚秒返回语义相关结果	通过
T13	安全评分	扫描标准站点	A+ - F 评分与中文修复建议正确	通过
T14	网络层检测	对自有域名执行 TLS/SPF 检测	证书状态与 DNS 记录判定正确	通过
T15	源码审计	上传含危险调用的样本	污点分析标记高危数据流	通过
T16	登录爆破门禁	不勾选授权声明	无法启动爆破任务	通过
T17	批量与导出	批量 3 站点并导出宽表 CSV	进度正常，导出 78 列 CSV 正确	通过

编号	测试项	测试方法	预期结果	结论
T18	历史 diff	同站两次扫描做对比	正确输出新增/消失/版本变化	通过
T19	扫描历史检索	按技术名检索历史	返回全部使用该技术的站点	通过
T20	Token 鉴权	无 Token 调用受保护 API	401 拒绝	通过

测试结果汇总如表 5-3 所示。全部 49 项单元测试与 32 项冒烟测试通过；真实站点实测中，演示站点标准识别 5.7 秒检出 13 项技术（含通过内联脚本指纹检出的 Microsoft Clarity），与人工核对结果一致；自构造泄露样本 4 项全部命中，干净站点零误报。

表 5-3 测试结果汇总

测试层次	数量	结果
单元/集成测试（unittest）	49 项	全部通过
API 冒烟测试（页面/参数/边界/导出）	32 项	全部通过
真实站点实测（演示站点标准识别）	13 项技术	识别正确，5.7s 完成
靶场回归（构造泄露样本）	4 项	全部命中，干净站零误报

性能维度的实测明细如表 5-4 所示。

表 5-4 性能实测明细

场景	目标 / 条件	耗时	结果
快速识别	演示站点，快速等级	约 5.7s	13 项技术，安全头评分 F
情报检索	关键词 nginx	<1s	命中相关情报并按语义排序
批量扫描	3 站点队列	≤20s	逐条完成，进度实时
源码审计	含危险调用的样本包	<3s	16 规则 + 污点分析报告

49 项单元测试按模块的分布如表 5-4 所示，核心逻辑（校验、匹配、比较、解析）的测试覆盖最厚；32 项 API 冒烟测试的分类统计如表 5-5 所示。

表 5-4 单元测试模块分布

被测模块	用例数	覆盖要点
target（目标校验）	8	协议白名单、内网/环回拒绝、DNS 解析校验
version_cmp（版本比较）	10	区间语法、通配、边界值

被测模块	用例数	覆盖要点
checks / select_nuclei (验证引擎)	9	软 404 基线、组匹配、语义路由优先级
evidence / detectors (解析与指纹)	8	meta/脚本/DOM 证据、边界匹配、去重合并
vuln / registry (情报与知识库)	8	三级判定、KEV 命中、模糊检索
storage / exporters (存储与导出)	6	落库完整性、四种导出格式

表 5-5 API 冒烟测试分类统计

分类	项数	代表用例
页面可达性	9	全部页面返回 200 且关键元素存在
参数校验	8	空 URL / 非 http 协议 / 内网地址全部拒绝
边界条件	6	404 路由、超长输入、重复提交
导出格式	4	JSON / 明细 CSV / 宽表 CSV / HTML 报告内容校验
统计与检索	5	/api/stats、类别表、情报检索结果正确性

测试中发现并修复的典型问题包括：扫描级扩展信息被子模块整体覆盖导致报告缺项、嵌于前端框架流式载荷中的内联脚本组件漏检（补充正文正则指纹解决）、中文 Windows 下 nslookup 输出编码不一致导致邮件安全检测误报（按 GBK 解码并逐行解析）、情报多来源合并时来源标记重复追加等，修复后均已完成回归验证。

测试阶段发现的缺陷均按「现象 → 根因 → 修复 → 回归」流程跟踪，典型缺陷记录如表 5-4 所示。

表 5-4 典型缺陷跟踪记录

缺陷现象	根因分析	修复措施	回归
报告缺少数个扩展模块的数据	扫描级 extras 被后执行模块整体覆盖	模块执行完后合并 extras（防覆盖）	通过
内联加载的分析脚本漏检	组件地址嵌在前端框架流式载荷中而非标签内	新增正文正则指纹	检出正常
三个域名的邮件检测结果雷同	中文系统下 nslookup 输出编码与续行解析错误	GBK 解码 + 按记录段收集续行	判定正确
目录探测 403 行无路径显示	前端字段名与后端不一致	统一 extras 渲染器并剔除响应体	回显正常

以量化视角汇总系统的最终成果，如表 6-1 所示——所有指标都可在系统中直接查验或复现。

表 5-6 成果指标汇总

成果维度	指标	数值
------	----	----

成果维度	指标	数值
检测能力	产品指纹 / 检测类别	2850+ / 78
验证能力	社区模板 / 自研无害检测项	2613 / 42
情报能力	漏洞情报 / 版本区间 / KEV	11024 / 1323 / 1695
工程质量	单元测试 / API 冒烟	49 / 32 (全部通过)
性能表现	标准识别耗时 (单站)	约 5.7 秒
代码规模	核心包 + 应用层 (不含前端)	约 5000 行 Python

5.3 测试结论

综合三层测试结果可以得出结论：系统各功能模块在正常与异常输入下行为均符合设计预期；面向对象架构的可扩展性在实践中得到验证（多个检测器与验证引擎的接入均未修改引擎）；合规约束真实有效，全部负向用例（内网目标、未授权操作、无 Token 访问）均被正确拒绝；识别能力经真实站点与构造样本双重验证，误报率控制在可用水平。系统达到设计目标，具备交付使用的条件。

6 总结

本实训完成了一个功能完整、可直接使用的验证型 Web 漏洞扫描器。系统的主要优点：（1）面向对象结构清晰，检测器继承体系使功能扩展成本极低，实训期间三次能力升级均未修改引擎代码；（2）「指纹 → 验证 → 情报」三级链路完整，三级判定机制把误报控制在了可用水平；（3）工程化程度高，具备异步任务、进度反馈、批量扫描、历史对比与多格式导出，不是演示玩具而是真实可用的工具；（4）合规设计内建于代码，SSRF 防护、限速与授权门禁不依赖使用者自觉。

系统也存在明显不足：（1）受 Python 线程与全局限速约束，吞吐量与 Go 系引擎存在量级差距，批量场景下偏慢；（2）依赖 requests 栈，无法覆盖需要浏览器渲染才能加载的组件，也缺少盲注类漏洞必需的 OOB 回连检测；（3）DAST 与污点分析为教学级深度，距工业工具仍有差距；（4）漏洞情报的时效性依赖定期运行更新脚本。

后续改进措施：优先为验证引擎增加响应内容抽取（extractor），把验证环节捕获的版本号回灌情报判定，进一步扩大「确认受影响」的覆盖面；其次按路径聚类合并模板请求以提升吞吐；中期引入轻量回连服务器以支持盲注检测；同时扩大靶场回归集，用可重复的断言持续校准误报率。

回顾整个开发过程，我们的收获与体会主要有三点。其一，面向对象不是语法条文而是设计直觉：当初把检测器设计成继承树纯属「教科书式」选择，后来三次大规模能力升级都因此受益，才真正理解了「对扩展开放、对修改关闭」的工程含义。其二，爬虫技术是把双刃剑：requests 与 BeautifulSoup 让采集变得简单，但正因为简单，才更需要限速、身份标识与目标校验这些「刹车」——工具的能力边界必须由开发者主动画出来。其三，真实数据规模会暴露一切假设：2481 条社区指纹入库后暴露了短词误报、性能劣化等一系列小样本阶段看不见的问题，逼着我们补齐边界匹配、trgm 索引与语义缓存设计。

通过本次实训，我们完整经历了从需求分析、面向对象设计、编码实现到系统测试的工程全过程：封装让数据有了边界，继承与多态让能力可以生长，组合让系统可以像积木一样搭建；requests 与 BeautifulSoup 则是整个系统的眼睛。更重要的是，我们体会到安全工具「能力越大、克制越多」的合规责任——扫描器的每一行代码都必须同时回答「能做什么」和「不做什么」。这些认识将伴随我们后续的专业学习与工程实践。